Costeñol | Guía breve

Resumen para estudiar la aplicación implementada en Python.
Actualizado al 6 de noviembre de 2026. Extensión predeterminada: .pqek.

El proyecto reconoce código Costeñol, comprueba su estructura y significado, y lo ejecuta desde consola o una interfaz Tkinter. Aunque se llama compilador en el curso, la ejecución actual interpreta el árbol sintáctico: no genera binarios ni código máquina.

Las cinco partes de la aplicación

Archivo	Qué hace
lexer.py	Reconoce tokens y conserva línea y columna.
parser.py	Ordena los tokens en un árbol sintáctico (AST).
semantico.py	Valida declaraciones, tipos e inicialización.
interprete.py	Calcula, captura datos, muestra mensajes y ejecuta ramas.
interfaz.py	Editor, archivos, análisis y ejecución con diálogos.

Organización actual

```
costenol/
  lexer.py, parser.py, semantico.py
  interprete.py, interfaz.py
  test_*.py
  README.md
  ejemplos/
    ejemplo_ejecucion.pqek
    ejemplo_concatenacion.pqek
    otros ejemplos .pqek y .cost
```

Los archivos de práctica se trasladaron a ejemplos sin cambiar su contenido. Algunos pueden conservar errores que hayas añadido al estudiar. Las funciones y métodos tienen documentación interna en español, visible en VS Code.

Lenguaje y concatenación

```
num1 Entero;
num2 Entero;
res Entero;
num1 = Captura.Entero();
num2 = Captura.Entero();
res = num1 + num2;
Mensaje.Texto("El resultado es: " + res);
```

Con entradas 5 y 3 se muestra **El resultado es: 8** en una sola línea. El operador + suma números o concatena si uno de sus operandos es Texto y el otro Texto, Entero o Real. No concatena Logico.

```
Mensaje.Texto("Total: " + (2 + 3)); // Total: 5 //
Mensaje.Texto("Total: " + 2 + 3);   // Total: 23 //
```

Los + se agrupan por la izquierda; usa paréntesis para calcular antes de concatenar. Los reales se convierten a texto con coma decimal. La concatenación también se puede guardar en una variable Texto.

Reglas esenciales

Tipos: Texto, Entero, Real y Logico. La declaración escribe nombre y tipo. Declaraciones, asignaciones y mensajes llevan ;. Las mayúsculas cuentan: Si y si son diferentes. Los comentarios se abren y cierran con // en la misma línea; las cadenas usan comillas dobles rectas. Los reales usan coma: 3,5.

```
edad Entero;
edad = Captura.Entero();
Si (edad >= 18)
    Mensaje.Texto("Mayor de edad");
Sino
    Mensaje.Texto("Menor de edad");
FinSi
```

Sino es opcional y se permiten condicionales anidados. No hay punto y coma después de Si, Sino ni FinSi. Verdadero y Falso son respuestas de Captura.Logico(), no literales reservados del código fuente. Puedes obtener Logico mediante comparaciones.

Cómo se procesa el código

1. Lexer: reconocer

tokenizar recorre el texto de izquierda a derecha. Omite espacios y comentarios, reconoce cadenas y usa patrones para números e identificadores. Consulta PALABRAS y SIMBOLOS. Prueba reales antes de enteros y símbolos dobles antes de simples. Cada Token guarda tipo, lexema, línea y columna.

```
edad = 20;
IDENTIFICADOR | ASIGNACION | NUM_ENTERO | PUNTO_Y_COMA
```

2. Parser: organizar

aceptar consume un token si coincide; exigir lo requiere o informa un error. instruccion reconoce declaraciones, asignaciones, mensajes y condicionales. bloque reúne instrucciones hasta un cierre. El AST usa diccionarios y listas; todavía no contiene resultados calculados.

expresion, suma, producto, unaria y primaria establecen la prioridad. En 2 + 3 * 4, el árbol agrupa 3 * 4 antes de la suma. No se aceptan comparaciones encadenadas como a < b < c.

3. Semántica: comprobar significado

La tabla de símbolos registra nombre, tipo, ubicación e inicialización. Una variable debe declararse y recibir valor antes de leerse. No se repiten nombres visibles. Se permite Entero hacia Real; no Real hacia Entero. La división produce Real. Los comparadores producen Logico y Si exige una condición de ese tipo.

Las declaraciones dentro de cada rama son locales. Una variable exterior queda inicializada al salir si ya tenía valor o se asigna en ambas ramas. Se revisan las dos ramas, incluso si una condición parece constante.

4. Intérprete: ejecutar

ejecutar valida todo antes de producir efectos. evaluar calcula las expresiones; bloque ejecuta instrucciones y elige una sola rama. El entorno guarda valores. Las funciones de entrada y salida se pueden sustituir: la consola usa input/print y la interfaz usa diálogos y panel de resultados. No se usa eval ni exec.

Uso de consola e interfaz

Ejecuta los comandos desde la carpeta costenol. Requiere Python 3.10 o posterior; la ventana necesita Tcl/Tk. No requiere paquetes externos de pip. Si tu lanzador es py, úsalo en lugar de python.

```
python interfaz.py
python lexer.py ejemplos/ejemplo.pqek
python parser.py ejemplos/ejemplo_sintactico.pqek --arbol
python semantico.py ejemplos/ejemplo_semantico.pqek --simbolos
python interprete.py ejemplos/ejemplo_ejecucion.pqek
```

Interfaz

Abrir: selecciona un archivo dentro de ejemplos. Se admiten .pqek, .cost y otros textos UTF-8. **Guardar como:** propone .pqek. **Guardar:** conserva la ruta actual. Se pregunta antes de descartar cambios.

F5 / Analizar: comprueba léxico, sintaxis y semántica sin capturar datos. **F6 / Ejecutar:** valida y ejecuta el texto actual aunque no esté guardado. Los errores aparecen con categoría, línea y columna; la línea se resalta. Ctrl+N crea, Ctrl+O abre y Ctrl+S guarda.

Capturas y salida

Tipo	Entrada válida
Texto	Texto incluso vacío; conserva espacios.
Entero	20 o -5; sin parte decimal.
Real	12 o -3,5; sin punto decimal.
Logico	Verdadero o Falso; sin distinguir mayúsculas.

En el ejemplo de ejecución introduce Ana, 20, 12,5 y Verdadero, una respuesta por captura. La salida incluirá el saludo, Mayor de edad, Asistencia confirmada y total 25,0. Cada Mensaje.Texto agrega un salto de línea. Cancelar o introducir un dato inválido detiene la ejecución.

Errores, pruebas y repaso

Error	Caso para reconocerlo
Léxico	edad = @; contiene un carácter desconocido.
Sintáctico	edad = ; carece de una expresión.
Semántico	edad Entero; edad = "Hola"; mezcla tipos incompatibles.
De ejecución	Mensaje.Texto(1 / 0); intenta dividir entre cero.

Se presenta el primer error; corrige y analiza nuevamente. La ubicación señala dónde se detectó el problema y puede ser el token posterior a una omisión. Si la ejecución ya mostró mensajes antes de fallar, estos no se revierten.

```
python -m unittest discover -p "test_*.py" -v
```

Las pruebas cubren parser, tipos, inicialización, alcance, capturas, operaciones, concatenación, errores e integración gráfica. La verificación registrada tras estos cambios debe leerse con las pruebas gráficas omitidas si Tcl/Tk no inicia. Una prueba skipped no equivale a una comprobación realizada.

Preguntas para estudiar

¿Token y lexema? Lexema es el texto; token añade categoría y ubicación.
¿AST y resultado? El AST describe la estructura; el intérprete calcula el valor.
¿Símbolos y entorno? La tabla valida nombres y tipos; el entorno guarda valores.
¿Analizar garantiza ejecutar? No: una captura futura o división entre cero puede fallar.
¿Por qué validar antes? Evita pedir datos o mostrar resultados de un programa estáticamente inválido.

Límites y decisiones actuales

No hay ciclos, funciones de usuario, arreglos ni operadores lógicos compuestos. La interfaz es síncrona, sin depurador ni botón Detener. Usa float para reales y puede haber redondeos. La revisión visual de Tkinter sigue pendiente en el entorno de desarrollo. Comparadores, alcance local, concatenación y conversiones son reglas adoptadas por el equipo.

Base documental: código de los cinco módulos, sus docstrings y pruebas. Esta guía breve actualiza las reglas de concatenación, extensión y organización respecto a la guía anterior. Orden de estudio: lexer, parser, semántica, intérprete e interfaz.